Step 1 — The Match Report and the Matcher — Hints

> Back to the step · Hint 1 · Hint 2 · Solution

- Hint 1 — the shapes, and the trick that makes it testable
- Hint 2 — match_skills, line by line

Hint 1 — the shapes, and the trick that makes it testable

Two models in `src/schemas.py`:

- `RequiredSkills(skills: List[str] = [])` — what the job extractor returns.
- `MatchReport(score: int, matched: List[str] = [], missing: List[str] = [])` — the result.

The trick that makes the matcher testable with no model: `match_skills` takes a **`distance` function** as an argument, not a Chroma collection. In tests you pass a fake `distance(skill)` that returns made-up numbers; in Step 3 you pass a real one that searches the resume. The matcher doesn't know or care which.

Hint 2 — match_skills, line by line

```
match_skills(required, distance, cutoff=0.8):
    matched, missing = [], []
    for skill in required:
        if distance(skill) <= cutoff:   # close enough in the resume
            matched.append(skill)
        else:
            missing.append(skill)
    total = len(required)
    if total == 0:
        score = 0                       # never divide by zero
    else:
        score = round(100 * len(matched) / total)
    return MatchReport(score=score, matched=matched, missing=missing)
```

Test it with a fake `distance`: `{"python": 0.1, "docker": 0.3, ...}.get(skill.lower(), 1.5)` — the covered skills return small numbers, the rest return 1.5 (over the 0.8 cutoff). Then `["Python","Docker","AWS","Kubernetes","Leadership"]` gives 4 matched, 1 missing, score 80 — and `[]` gives score 0, not a crash. The whole file set is in the Solution.